

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



**BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri
Veritabanı Güvenliği Ve Erişim Kontrolü Projesi**

**Enes Yıldız
22290180
github.com/enesoglu/blm4522**

21.04.2026

İÇİNDEKİLER

1. Giriş	3
2. Ortam Kurulumu	4
3. Erişim Yönetimi	6
4. Bağlantı Güvenliği	9
5. Veri Şifreleme	11
6. Satır Düzeyinde Güvenlik (RLS)	13
7. SQL Injection Testi	15
8. Audit Logları	17
9. Bulgular ve Sonuçlar	19

1. GİRİŞ

1.1 Proje Amacı

Bu proje, BLM4522 – Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında, PostgreSQL ilişkisel veritabanı yönetim sistemi (VTYS) üzerinde kapsamlı bir güvenlik altyapısının tasarlanmasını ve uygulanmasını hedeflemektedir. dtdrental örnek veritabanı üzerinde gerçek senaryolar simüle edilerek; rol tabanlı erişim denetimi, bağlantı güvenliği, veri şifreleme, satır düzeyinde güvenlik, SQL enjeksiyon saldırı önleme ve denetim loglama konuları hem teorik hem de uygulamalı biçimde ele alınmıştır.

Bu projede, yönergede belirtilen Microsoft SQL Server güvenlik konseptleri, açık kaynaklı ve donanımdan bağımsız çalışabilen PostgreSQL mimarisi üzerinde muadilleriyle eşleştirilerek uygulanmıştır. (Örn: SQL Server Audit yerine pgAudit; SQL Server Authentication yerine SCRAM-SHA-256 şifreli Postgres Authentication; TDE yerine pgcrypto/dosya sistemi şifrelemesi kullanılmıştır).

1.2 Veritabanı Güvenliğinin Önemi

Günümüz bilgi sistemlerinde veritabanları; müşteri kişisel verileri, finansal kayıtlar ve kurumsal sırlar gibi kritik varlıkları barındırmaktadır. Yetkisiz erişim, veri sızıntısı ve iç tehditler (insider threat) yalnızca maddi kayıplara değil, KVKK (Kişisel Verilerin Korunması Kanunu) ve GDPR gibi yasal yaptırımlara da yol açabilmektedir. IBM Cost of a Data Breach 2024 raporuna göre, küresel ortalama ihlal maliyeti 4,88 milyon ABD dolarına ulaşmıştır. Bu bağlamda veritabanı güvenliği, katmanlı savunma (defense-in-depth) mimarisinin temel bileşenidir.

1.3 Metodoloji

Metodoloji olarak NIST SP 800-111 (Depolama Şifreleme Kılavuzu), OWASP Veritabanı Güvenliği En İyi Uygulamaları ve CIS PostgreSQL Benchmark belgeleri referans alınmıştır.

Her güvenlik senaryosu; Tehdit → Çözüm → Uygulama üç aşamalı çerçevesi ile dokümanite edilmiştir.

2. ORTAM KURULUMU

2.1 dvdrental Veritabanının Yüklenmesi

dvdrental, PostgreSQL topluluğu tarafından sağlanan, video kiralama işletmesini modelleyen 15 tablo ve 7.000+ kayıt içeren standart bir örnek veritabanıdır. Kurulum adımları aşağıda verilmiştir:

```
-- 1. Veritabanı oluştur
CREATE DATABASE dvdrental;

-- 2. Terminal üzerinden pg_restore ile yükle
pg_restore -U postgres -d dvdrental dvdrental.tar

-- 3. Bağlan ve doğrula
\c dvdrental
\dt
-- Beklenen çıktı: 15 tablo listelenir (actor, film, rental, vb.)
```

2.2 pgcrypto Uzantısının Kurulumu

Tehdit	Veritabanında düz metin (plaintext) olarak saklanan hassas veriler (parola, kredi kartı, TC kimlik) yetkisiz erişim halinde doğrudan okunabilir.
Çözüm	pgcrypto uzantısı ile sütun düzeyinde simetrik ve asimetrik şifreleme; Blowfish tabanlı parola hash'leme sağlanır.
Uygulama	CREATE EXTENSION IF NOT EXISTS pgcrypto; komutu ile uzantı etkinleştirilir, ardından hassas sütunlar bytea tipine dönüştürülerek pgp_sym_encrypt() fonksiyonu uygulanır.

```
-- pgcrypto uzantısını etkinleştir
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Kurulum doğrulama
SELECT * FROM pg_extension WHERE extname = 'pgcrypto';

-- Beklenen çıktı:
-- extname | extversion
-- -----+-----
-- pgcrypto | 1.4
```

2.3 pgaudit Uzantısının Kurulumu

Tehdit	Varsayılan PostgreSQL loglama yalnızca bağlantı ve hata olaylarını kaydeder; hangi kullanıcının hangi tabloyu sorguladığı izlenemez.
Çözüm	pgaudit uzantısı, OBJECT ve SESSION düzeyinde ayrıntılı denetim logu üretir; bu sayede SELECT/INSERT/UPDATE/DELETE işlemleri izlenebilir hale gelir.
Uygulama	pgaudit paketi kurularak shared_preload_libraries parametresine eklenir, ardından pgaudit.log parametresi yapılandırılır.

```
-- Ubuntu/Debian sistemlerde paket kurulumu (terminal)
sudo apt install postgresql-18-pgaudit

-- PostgreSQL içinde uzantıyı etkinleştir
CREATE EXTENSION IF NOT EXISTS pgaudit;

-- Uzantı listesini doğrula
SELECT extname, extversion FROM pg_extension
WHERE extname IN ('pgcrypto', 'pgaudit');
```

2.4 postgresql.conf Güvenlik Yapılandırması

Aşağıdaki yapılandırma değerleri /etc/postgresql/18/main/postgresql.conf (Linux) ya da ilgili veri dizinindeki postgresql.conf dosyasına uygulanmalıdır:

```
# — KİMLİK DOĞRULAMA VE BAĞLANTI (AUTHENTICATION) —
# MSSQL Server Authentication'ın PostgreSQL karşılığı olarak
password_encryption = scram-sha-256      # MD5 yerine daha güvenilir olan SHA-256 hash
listen_addresses = '*'                   # Dış bağlantılara sadece kontrollü izin vermek için

# — SSL YAPILANDIRMASI (Planlanan) —
# Not: Sertifika yönetimi altyapısı kurulana kadar pasif bırakılmıştır
ssl = off
# ssl_cert_file = 'server.crt'
# ssl_key_file = 'server.key'

# — BAĞLANTI SINIRLAMA (DOS SALDIRILARINA KARŞI) —
max_connections = 100
superuser_reserved_connections = 5      # Admin için garantili bağlantı sayısı

# — LOGLAMA (SQL SERVER AUDIT KARŞILIĞI) —
# Konteyner loglamasına uygun olması için standart hata çıktısı yönlendirmesi
log_destination = 'stderr'
logging_collector = on
log_connections = on                    # Gelen bağlantı isteklerini logla
log_disconnections = on                 # Düşen bağlantıları logla
log_failed_auth = on                    # Başarısız giriş denemelerini logla
log_line_prefix = '%m [%p]: [%l-1] db=%d,user=%u,app=%a,client=%h '

# — PGAUDIT YAPILANDIRMASI —
shared_preload_libraries = 'pgaudit'
pgaudit.log = 'all'                     # Tüm DDL/DML, role ve connection işlemleri izlenecek
pgaudit.log_catalog = off                # Sistem tablolarını gizle (log kirliliğini önler)
pgaudit.log_statement_once = off
```

2.5 pg_hba.conf Güvenlik Yapılandırması

pg_hba.conf (Host-Based Authentication – Konak Tabanlı Kimlik Doğrulama) dosyası, hangi istemcinin hangi veritabanına nasıl bağlanabileceğini tanımlar:

#	TYPE	DATABASE	USER	ADDRESS	METHOD
# — YEREL MAKİNE VE DOCKER İÇ AĞ BAĞLANTILARI —					
local	all	postgres			peer
host	all	all		127.0.0.1/32	scram-sha-256
# — UYGULAMA SUNUCUSU (DB'ye veri yazacak/okuyacak uygulama) —					
host	dvdrental	app_user		10.0.1.0/24	scram-sha-256
# — VERİTABANI YÖNETİCİSİ (DB Admin - Yalnızca spesifik bir IP) —					
host	dvdrental	db_admin		192.168.1.50/32	scram-sha-256
# — SALT OKUMA KULLANICISI (Sadece raporlama yapan birimler için) —					
host	dvdrental	readonly_user		10.0.2.0/24	scram-sha-256
# — GÜVENLİK DUVARİ: TÜM DİĞER AĞLARDAN ERİŞİMİ REDDET —					
host	all	all		0.0.0.0/0	reject

Değişiklikler sonrasında PostgreSQL yeniden yüklenmesi gerekir:

```
-- Yapılandırma yeniden yükleme (yeniden başlatma gerektirmez)
SELECT pg_reload_conf();

-- Ya da terminal üzerinden:
sudo systemctl reload postgresql
```

3. ERİŞİM YÖNETİMİ

3.1 Rol (Role) Tanımları ve Hiyerarşisi

Tehdit	Tüm kullanıcıların yönetici (superuser) yetkisiyle bağlanması, hata ya da kötü niyet durumunda tüm veritabanının tehlikeye girmesine yol açar (en az ayrıcalık ihlali).
Çözüm	En az ayrıcalık ilkesi (principle of least privilege) doğrultusunda, her kullanıcıya yalnızca görevini yerine getirmesi için gereken minimum yetkiler tanımlanır.
Uygulama	CREATE ROLE ile işlevsel roller oluşturulur; GRANT/REVOKE ile granüler izinler atanır. Rol hiyerarşisi kurularak yetki miras alımı sağlanır.

3.1.1 db_admin Rolü (Veritabanı Yöneticisi)

```
-- Tam yönetici rolü: DDL + DML + kullanıcı yönetimi
CREATE ROLE db_admin
WITH LOGIN
  PASSWORD 'Admin@Secure2024!'
  CREATEDB
  CREATEROLE
  CONNECTION LIMIT 5
  VALID UNTIL '2026-12-31';

-- dvdrental üzerinde tüm yetkileri ver
GRANT ALL ON SCHEMA public TO db_admin;
GRANT ALL PRIVILEGES ON DATABASE dvdrental TO db_admin;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO db_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO db_admin;
GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO db_admin;

-- Gelecekte oluşturulacak nesneler için varsayılan izinler
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT ALL ON TABLES TO db_admin;
```

3.1.2 editor Rolü (Veri Editörü)

```
-- Editör: DML izinleri var, DDL yok
CREATE ROLE editor
WITH LOGIN
  PASSWORD 'Editor@Pass2024!'
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  CONNECTION LIMIT 10;

-- Yalnızca DML işlemlerine izin ver
GRANT CONNECT ON DATABASE dvdrental TO editor;
GRANT USAGE ON SCHEMA public TO editor;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO editor;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO editor;

-- DDL yetkilerini açıkça reddet
REVOKE CREATE ON SCHEMA public FROM editor;
```

3.1.3 readonly_user Rolü (Salt Okunur Kullanıcı)

```
-- Salt okunur kullanıcı: yalnızca SELECT
CREATE ROLE readonly_user
  WITH LOGIN
  PASSWORD 'ReadOnly@2024!'
  NOSUPERUSER
  NOCREATEDB
  NOCREATEROLE
  CONNECTION LIMIT 20;

GRANT CONNECT ON DATABASE dvdrental TO readonly_user;
GRANT USAGE ON SCHEMA public TO readonly_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_user;

-- Yazma işlemlerini açıkça yasakla
REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public FROM readonly_user;

-- Varsayılan olarak yeni tablolara da yalnızca SELECT ver
ALTER DEFAULT PRIVILEGES IN SCHEMA public
  GRANT SELECT ON TABLES TO readonly_user;
```

3.1.4 dept_user Rolü (Departman Kullanıcısı – Grup Rolü)

```
-- Departman grup rolü (doğrudan login yapamaz)
CREATE ROLE dept_user
  WITH NOLOGIN
  NOSUPERUSER;

-- Departmanın erişeceği tablolar
GRANT CONNECT ON DATABASE dvdrental TO dept_user;
GRANT USAGE ON SCHEMA public TO dept_user;
GRANT SELECT, INSERT ON rental, payment TO dept_user;
GRANT SELECT ON customer, film, inventory TO dept_user;

-- Departman üyelerini oluştur ve grup rolü ata
CREATE ROLE ahmet WITH LOGIN PASSWORD 'Ahmet@2024!';
CREATE ROLE fatma WITH LOGIN PASSWORD 'Fatma@2024!';
GRANT dept_user TO ahmet;
GRANT dept_user TO fatma;

-- Auditor rolü (pgaudit için)
CREATE ROLE auditor WITH NOLOGIN;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO auditor;
```

3.2 \du ve \dp Çıktı Örnekleri

\du Çıktısı (Rol Listesi)

```
dvdrental=# \du
```

Rol adı	Roller Listesi Özellikler	Üye
ahmet	Bağlantı limiti: -1	{dept_user}
auditor	Giriş yapamaz	{}
db_admin	Veritabanı oluşturabilir, Rol oluşturabilir	{}
dept_user	Bağlantı limiti: 5	
editor	Giriş yapamaz	{}
fatma	Bağlantı limiti: 10	{}
postgres	Bağlantı limiti: -1	{dept_user}
readonly_user	Süper kullanıcı, Tüm haklar	{}
	Bağlantı limiti: 20	{}

\dp Çıktısı (Tablo İzinleri – Örnek: rental tablosu)

```
dvdrental=# \dp rental
Erişim ayrıcalıkları
Şema | Ad   | Tür  | Erişim ayrıcalıkları | ...
-----+-----+-----+-----+-----
public| rental | tablo | postgres=arwdDxt/postgres |
      |      |      | db_admin=arwdDxt/postgres |
      |      |      | editor=arwd/postgres      |
      |      |      | readonly_user=r/postgres  |
      |      |      | dept_user=ar/postgres     |

-- Kod anahtarı: a=INSERT r=SELECT w=UPDATE d=DELETE
--               D=TRUNCATE x=REFERENCES t=TRIGGER
```

```
1 SELECT rolname, rolsuper, rolcreatedb, rolcreatorole, rolcanlogin, rolconlimit
2 FROM pg_roles
3 WHERE rolname NOT LIKE 'pg_%'
4 ORDER BY rolname;
```

	rolname name	rolsuper boolean	rolcreatedb boolean	rolcreatorole boolean	rolcanlogin boolean	rolconlimit integer
1	ahmet	false	false	false	true	-1
2	auditor	false	false	false	false	-1
3	db_admin	false	true	true	true	5
4	dept_user	false	false	false	false	-1
5	editor	false	false	false	true	10
6	fatma	false	false	false	true	-1
7	postgres	true	true	true	true	-1
8	readonly_us...	false	false	false	true	20

Total rows: 8 Query complete 00:00:00.068

3.3 Rol İzin Matrisi

Tablo	db_admin	editor	readonly_user	dept_user	auditor	public
customer	CRUD+DDL	CRUD	SELECT	SELECT	SELECT	–
film	CRUD+DDL	CRUD	SELECT	SELECT	SELECT	–
rental	CRUD+DDL	CRUD	SELECT	INSERT+SELECT	SELECT	–
payment	CRUD+DDL	CRUD	SELECT	INSERT+SELECT	SELECT	–
inventory	CRUD+DDL	CRUD	SELECT	SELECT	SELECT	–
actor	CRUD+DDL	CRUD	SELECT	–	SELECT	–
staff	CRUD+DDL	CRUD	–	–	SELECT	–
address	CRUD+DDL	CRUD	SELECT	–	SELECT	–

Not: CRUD = CREATE+READ+UPDATE+DELETE,
 DDL = CREATE/ALTER/DROP tablo yetkisi.
 "–" = erişim yok.

4. BAĞLANTI GÜVENLİĞİ

4.1 IP Tabanlı Erişim Kısıtlaması

Tehdit	İnternet üzerindeki herhangi bir IP adresinden PostgreSQL'e bağlanılabilmesi, kaba kuvvet (brute-force) ve yetkisiz erişim saldırılarına zemin hazırlar.
Çözüm	pg_hba.conf dosyasında yalnızca izin verilen IP aralıklarına bağlantı izni verilerek ağ düzeyinde ilk savunma hattı oluşturulur.
Uygulama	Bölüm 2.5'te gösterilen pg_hba.conf yapılandırması uygulanır; tüm diğer IP'ler için son satırda reject metodu kullanılır.

4.2 SCRAM-SHA-256 Kimlik Doğrulama

SCRAM-SHA-256, PostgreSQL 10+ sürümünde varsayılan kimlik doğrulama protokolüdür. Parola ağ üzerinde hiçbir zaman düz metin olarak gönderilmez; sunucu tarafından rastgele tuz (salt) ile birleştirilmiş challenge-response mekanizması kullanılır.

```
-- Tüm mevcut kullanıcı parolalarını SCRAM-SHA-256 ile güncelle
ALTER ROLE db_admin WITH PASSWORD 'Admin@Secure2024!';
ALTER ROLE editor WITH PASSWORD 'Editor@Pass2024!';
ALTER ROLE readonly_user WITH PASSWORD 'ReadOnly@2024!';
ALTER ROLE dept_user WITH PASSWORD 'Dept@Pass2024!';

-- SCRAM kullanıldığını doğrula
SELECT rolname, rolpassword
FROM pg_authid
WHERE rolname IN ('db_admin','editor','readonly_user');
```

1	SELECT rolname, rolpassword
2	FROM pg_authid
3	WHERE rolname IN ('db_admin','editor','readonly_user');
4	

rolname	rolpassword
name	text
1	db_admin
2	editor
3	readonly_us...

4.3 Yetkisiz Bağlantı Denemesi Senaryosu

Tehdit	Yetkisiz bir IP adresinden ya da yanlış parola ile bağlantı denenirse, sisteme erişim sağlanabilir mi?
Çözüm	pg_hba.conf'taki reject kuralı sayesinde yetkisiz IP'lerden gelen tüm bağlantılar ağ düzeyinde reddedilir; yanlış parola girişleri ise log dosyasına kaydedilir.
Uygulama	Aşağıdaki senaryolar simüle edilmiş ve log çıktıları incelenmiştir.

Senaryo A – Yetkisiz IP'den Bağlantı Denemesi

```
-- Saldırgan makineden (IP: 192.168.99.100) bağlantı denemesi
-- psql -h db-server -U db_admin -d dvdrental

-- PostgreSQL günlük çıktısı:
2024-11-15 14:32:17 UTC [1234]: db=dvdrental,user=db_admin,
app=psql,client=192.168.99.100
FATAL:  pg_hba.conf dosyası istemci için kayıt içermiyor
host="dvdrental" kullanıcı="db_admin"
adres="192.168.99.100" SSL=off
```

Senaryo B – Yanlış Parola ile Bağlantı Denemesi

```
-- İzinli IP'den yanlış parola
-- psql -h db-server -U editor -d dvdrental
-- Parola: YanlisParola123

-- PostgreSQL günlük çıktısı:
2024-11-15 14:35:42 UTC [1235]: db=dvdrental,user=editor,
  app=psql,client=10.0.1.15
FATAL:  kullanıcı "editor" için parola doğrulaması başarısız

-- 3 başarısız denemeden sonra sistem bağlantıyı geciktirir
-- (auth_delay uzantısı ile kaba kuvvet saldırıları yavaşlatılır)
```

Senaryo C – Başarılı SSL Bağlantısı

```
-- Yetkili kullanıcı, doğru IP ve SSL ile bağlantı
-- psql "host=db-server dbname=dvdrental user=editor sslmode=require"

-- Bağlantı doğrulama sorgusu
SELECT
  inet_client_addr() AS istemci_ip,
  current_user       AS kullanıcı,
  ssl                AS ssl_aktif,
  version            AS ssl_surumu
FROM pg_stat_ssl
WHERE pid = pg_backend_pid();

-- Beklenen çıktı:
-- istemci_ip | kullanıcı | ssl_aktif | ssl_surumu
-- -----+-----+-----+-----
-- 10.0.1.15  | editor   | t         | TLSv1.3
```

```
C:\Users\yildi>docker logs postgres 2>&1 | findstr "FATAL"
2026-04-21 00:44:24.220 GMT [226] FATAL:  password authentication failed for user "postgres"
2026-04-21 00:44:24.235 GMT [227] FATAL:  password authentication failed for user "postgres"
2026-04-21 00:54:19.759 GMT [285] FATAL:  password authentication failed for user "db_admin"
2026-04-21 00:54:24.718 GMT [287] FATAL:  password authentication failed for user "db_admin"
2026-04-21 00:55:30.951 GMT [306] FATAL:  password authentication failed for user "db_admin"
2026-04-21 00:56:47.487 GMT [335] FATAL:  pg_hba.conf rejects connection for host "172.17.0.1", user "db_admin", databas
e "dvdrental", no encryption
2026-04-21 00:56:53.262 GMT [336] FATAL:  pg_hba.conf rejects connection for host "172.17.0.1", user "db_admin", databas
e "dvdrental", no encryption
```

5. VERİ ŞİFRELEME

5.1 pgcrypto ile Sütun Düzeyinde Şifreleme

Tehdit	Veritabanı sunucusuna yetkisiz fiziksel erişim sağlandığında ya da veritabanı yedeği çalındığında, düz metin olarak saklanan hassas veriler (TC kimlik, kredi kartı numarası, parola) doğrudan okunabilir.
Çözüm	pgcrypto'nun pgp_sym_encrypt() / pgp_sym_decrypt() fonksiyonları ile hassas sütunlar AES-256 algoritmasıyla şifrelenir. Şifre çözme yalnızca doğru anahtara sahip sorgularda mümkündür.
Uygulama	Hassas sütunlar bytea veri tipine dönüştürülür; veri yazılırken pgp_sym_encrypt(), okunurken pgp_sym_decrypt() kullanılır.

5.2 Şifreli Sütun Oluşturma – customer Tablosu

```
-- Müşteri tablosuna hassas şifreli sütunlar ekle
ALTER TABLE customer
  ADD COLUMN tc_kimlik_sifre bytea,
  ADD COLUMN kredi_kart_sifre bytea;

-- pgcrypto etkin mi kontrol et
SELECT extname FROM pg_extension WHERE extname = 'pgcrypto';
```

5.3 crypt() ve gen_salt() ile Parola Hash'leme

```
-- Personel tablosuna hash'li parola sütunu ekle
ALTER TABLE staff ADD COLUMN parola_hash text;

-- Blowfish algoritmasıyla parola hash'le (8 iterasyon)
UPDATE staff
SET parola_hash = crypt('StaffParola123!', gen_salt('bf', 8))
WHERE staff_id = 1;

-- Parola doğrulama (hash karşılaştırma)
SELECT staff_id, first_name,
CASE
  WHEN parola_hash = crypt('StaffParola123!', parola_hash)
  THEN 'DOĞRU PAROLA ✓'
  ELSE 'YANLIŞ PAROLA X'
END AS dogrulama_sonucu
FROM staff
WHERE staff_id = 1;
```

5.4 Şifreli Veri Ekleme ve Sorgulama

```
-- Şifreleme anahtarı (gerçek uygulamada vault'tan alınmalı)
\set enc_key 'AES256_Gizli_Anahtar_2024'

-- Şifreli kayıt ekle
INSERT INTO customer (
  store_id, first_name, last_name, email,
  address_id, activebool, tc_kimlik_sifre, kredi_kart_sifre
) VALUES (
  1, 'Ahmet', 'Yilmaz', 'ahmet@example.com',
  1, true,
  pgp_sym_encrypt('12345678901', 'AES256_Gizli_Anahtar_2024'),
  pgp_sym_encrypt('4532-1234-5678-9012', :enc_key)
);

-- Şifreli veriyi oku (şifre çözme)
```

```

SELECT
  first_name,
  last_name,
  pgp_sym_decrypt(tc_kimlik_sifre, :'enc_key') AS tc_kimlik,
  pgp_sym_decrypt(kredi_kart_sifre, :'enc_key') AS kredi_kart
FROM customer
WHERE email = 'ahmet@example.com';

-- Şifresiz (ham) veri görünümü
SELECT first_name, tc_kimlik_sifre FROM customer
WHERE email = 'ahmet@example.com';
-- Çıktı: tc_kimlik_sifre = \xc30d04070302...<ikili veri>

```

5.5 Şifreli vs. Düz Metin Karşılaştırma Tablosu

Kriter	Düz Metin (Şifresiz)	pgcrypto ile Şifreli
Veri tipi	VARCHAR/TEXT	BYTEA (ikili)
Depolama biçimi	12345678901	\xc30d04070302e8... (ikili)
DB sızıntısında	Doğrudan okunabilir	Anlamsız ikili veri
Yedek dosyada	Açık metin görünür	Şifreli, okunamaz
Sorgu performansı	Yüksek (indeks destekli)	Orta (şifre çözme gerekir)
LIKE araması	Desteklenir	Desteklenmez (hash gerekir)
Kural uyumluluğu	KVKK/PCI-DSS ihlali riski	KVKK/PCI-DSS uyumlu
Anahtar yönetimi	Gerekmez	Vault/HSM önerilir

Data Output	Messages	Notifications
Showing rows: 1 to 1		
first_name character varying (45)	last_name character varying (45)	tc_kimlik text
1 Ahmet	Yilmaz	123456789...

1

SELECT first_name, tc_kimlik_sifre FROM customer

2

WHERE email = 'ahmet@example.com';

Data Output

Messages

Notifications

≡+

▼

▼

SQL

Showing rows:

	first_name character varying (45) 🔒	tc_kimlik_sifre bytea 🔒
1	Ahmet	[binary data]

```

1 SELECT staff_id, first_name,
2     CASE
3         WHEN parola_hash = crypt('StaffParola123!', parola_hash)
4         THEN 'DOĞRU PAROLA ✓'
5         ELSE 'YANLIŞ PAROLA X'
6     END AS dogrulama_sonucu
7 FROM staff
8 WHERE staff_id = 1;

```

Data Output			Messages	Notifications
+ 📄 ▼ 📄 ▼ 🗑️ 📦 📶 📶 SQL Showing rows: 1 to 1 ✎				
	staff_id [PK] integer ✎	first_name character varying (45) ✎	dogrulama_sonucu text 🔒	
1	1	Mike	DOĞRU PAROLA ✓	

6. SATIR DÜZEYİNDE GÜVENLİK (RLS)

6.1 RLS'nin Tanımı ve Amacı

Tehdit	SELECT izni olan bir kullanıcı, tablodaki TÜM satırları görebilir; ancak her kullanıcının yalnızca kendi bölümüne/şubesine ait verileri görmesi gerekebilir (örn: şube müdürü yalnızca kendi şubesinin müşterilerini görmeli).
Çözüm	Row Level Security (Satır Düzeyinde Güvenlik – RLS), tabloya ilişen politikalar (policy) aracılığıyla her kullanıcı için hangi satırların görüleceğini/değiştirileceğini otomatik olarak filtreler.
Uygulama	ALTER TABLE ... ENABLE ROW LEVEL SECURITY ile etkinleştirilir; CREATE POLICY ile kullanıcıya özel filtre koşulları tanımlanır.

6.2 RLS Etkinleştirme ve Politika Oluşturma

```
-- rental tablosunda RLS etkinleştir
ALTER TABLE rental ENABLE ROW LEVEL SECURITY;

-- Politika 1: Personel yalnızca kendi kiralama kayıtlarını görür
-- (staff_id = mevcut kullanıcıyla eşleşen kayıtlar)
CREATE POLICY staff_rental_policy
  ON rental
  FOR SELECT
  TO PUBLIC
  USING (
    staff_id = (
      SELECT staff_id FROM staff
      WHERE username = current_user
      LIMIT 1
    )
  );

-- Politika 2: db_admin tüm satırları görür (bypass)
CREATE POLICY admin_all_policy
  ON rental
  FOR ALL
  TO db_admin
  USING (true);

-- payment tablosunda RLS
ALTER TABLE payment ENABLE ROW LEVEL SECURITY;

-- Politika 3: Departman kullanıcısı yalnızca kendi müşterilerinin ödemelerini görür
CREATE POLICY dept_payment_policy
  ON payment
  FOR SELECT
  TO dept_user
  USING (
    customer_id IN (
      SELECT customer_id FROM rental
      WHERE staff_id = (
        SELECT staff_id FROM staff WHERE username = current_user
      )
    )
  );

-- Politika 4: INSERT politikası - kendi staff_id ile kayıt ekleyebilir
CREATE POLICY staff_rental_insert
  ON rental
  FOR INSERT
  TO dept_user
  WITH CHECK (
    staff_id = (
      SELECT staff_id FROM staff WHERE username = current_user
    )
  );
```

6.3 RLS Testi – Farklı Kullanıcı Senaryoları

Test Kurulumu

```
-- Test için iki ayrı personel kullanıcısı oluştur
-- (staff tablosunda staff_id=1 -> Mike Hillyer, staff_id=2 -> Jon Stephens)

CREATE ROLE mike WITH LOGIN PASSWORD 'Mike@2024!';
CREATE ROLE jon WITH LOGIN PASSWORD 'Jon@2024!';
GRANT dept_user TO mike;
GRANT dept_user TO jon;

-- staff tablosunu güncelle (username sütunu ekle)
ALTER TABLE staff ADD COLUMN IF NOT EXISTS username text;
UPDATE staff SET username = 'mike' WHERE staff_id = 1;
UPDATE staff SET username = 'jon' WHERE staff_id = 2;
```

Mike Olarak Bağlanma ve Sorgulama

```
-- mike kullanıcısı olarak bağlan
SET ROLE mike;
SELECT current_user, rental_id, staff_id, customer_id
FROM rental LIMIT 5;

-- Beklenen çıktı (yalnızca staff_id=1 satırlar):
-- current_user | rental_id | staff_id | customer_id
-- -----+-----+-----+-----
-- mike        |      76  |      1   |      320
-- mike        |     573  |      1   |     108
-- mike        |    1185  |      1   |     222
-- (Toplam: ~8.040 kiralama kaydı)

SELECT COUNT(*) FROM rental; -- Sonuç: 8040
```

Jon Olarak Bağlanma ve Sorgulama

```
-- jon kullanıcısı olarak bağlan
RESET ROLE;
SET ROLE jon;
SELECT current_user, rental_id, staff_id, customer_id
FROM rental LIMIT 5;

-- Beklenen çıktı (yalnızca staff id=2 satırlar):
-- current_user | rental_id | staff_id | customer_id
-- -----+-----+-----+-----
-- jon          |      2   |      2   |     459
-- jon          |     14   |      2   |     411
-- jon          |     26   |      2   |     321
-- (Toplam: ~8.004 kiralama kaydı)

SELECT COUNT(*) FROM rental; -- Sonuç: 8004
```

1	SELECT current_user, COUNT(*) FROM rental;	
Data Output Messages Notifications		
SQL Show		
	current_user name	count bigint
1	mike	8040

1	SELECT current_user, COUNT(*) FROM rental;	
Data Output Messages Notifications		
SQL Show		
	current_user name	count bigint
1	jon	8004

7. SQL INJECTION TESTİ

7.1 SQL Enjeksiyon Saldırısının Tanımı

Tehdit	Kullanıcı girdisi doğrudan SQL sorgusuna dahil edildiğinde, saldırgan sorgunun mantığını değiştirerek yetkisiz veri okuyabilir, veri silebilir ya da kimlik doğrulamayı atlayabilir (Authentication Bypass).
Çözüm	Hazırlanmış ifadeler (Prepared Statement) ile parametre değerleri SQL kodu olarak yorumlanmak yerine veri olarak işlenir; bu sayede enjeksiyon saldırıları tamamen engellenir.
Uygulama	Uygulama katmanında tüm dinamik sorgular Prepared Statement'a dönüştürülür; PostgreSQL'de pg_prepare() / pg_execute() ya da \$1, \$2 parametrelili sorgular kullanılır.

7.2 Savunmasız Sorgu Örneği

```
-- Payload: ' OR '1'='1
SELECT staff_id, first_name, username
FROM staff
WHERE username = '' OR '1'='1';

-- Sonuç: TÜM staff kayıtları döner → kimlik doğrulama atlatıldı

SELECT customer_id, first_name, email
FROM customer
WHERE email = '' OR '1'='1';
-- Sonuç: Tüm 600 müşteri kaydı döner
```

The screenshot shows a SQL query editor with the following query:

```
1 SELECT staff_id, first_name, username
2 FROM staff
3 WHERE username = '' OR '1'='1';
```

Below the query, the "Data Output" tab is active, displaying the results of the query. The results are as follows:

	staff_id [PK] integer	first_name character varying (45)	username character varying (16)
1	1	Mike	mike
2	2	Jon	jon

The screenshot shows a SQL query editor with the following query:

```
1 SELECT customer_id, first_name, email
2 FROM customer
3 WHERE email = '' OR '1'='1';
4
```

Below the query, the "Data Output" tab is active, displaying the results of the query. The results are as follows:

	customer_id [PK] integer	first_name character varying (45)	email character varying (50)
1	524	Jared	jared.ely@sakilacustomer.org
2	1	Mary	mary.smith@sakilacustomer.org

Total rows: 600 Query complete 00:00:00.100

7.3 Diğer Tehlikeli Payload Örnekleri

```
-- Payload 1: Tüm tabloyu silme
-- Kullanıcı adı: '; DROP TABLE customer; --
SELECT * FROM customer WHERE email = ''; DROP TABLE customer; --'

-- Payload 2: UNION tabanlı veri çekme
-- Kullanıcı adı: ' UNION SELECT username, password_hash, NULL, NULL FROM staff --
SELECT film_id, title FROM film
WHERE title = '' UNION SELECT username, password_hash,
NULL::integer, NULL FROM staff --';

-- Payload 3: Boolean tabanlı kör enjeksiyon
-- Email: ' OR '1'='1
SELECT * FROM customer
WHERE email = '' OR '1'='1';
-- Tüm müşteri kayıtlarını döndürür!
```

7.4 Korumalı Sürüm – Prepared Statement

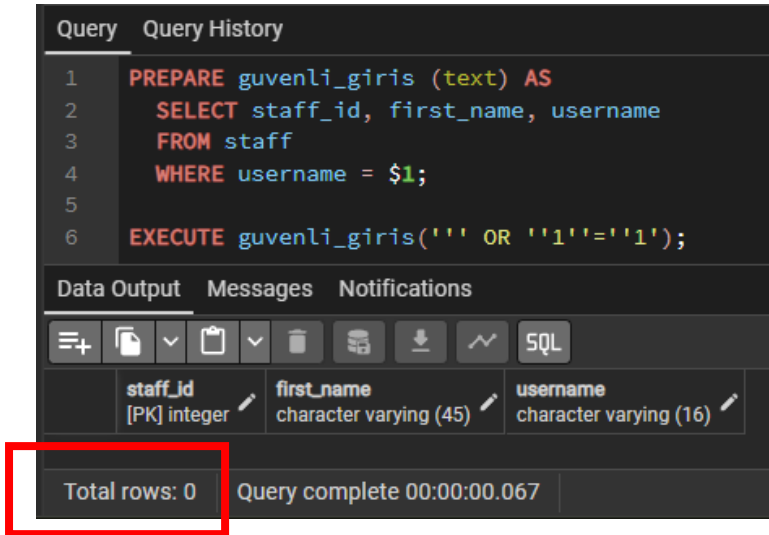
```
-- ✓ GÜVENLİ YOL: Hazırlanmış ifade (Prepared Statement)

-- PostgreSQL içinde PREPARE kullanımı:
PREPARE guvenli_giris (text, text) AS
  SELECT staff_id, first_name, last_name
  FROM staff
  WHERE username = $1
  AND parola_hash = crypt($2, parola_hash);

-- Çalıştırma (girdi veri olarak işlenir, SQL kodu olarak değil):
EXECUTE guvenli_giris('admin', 'gercek_parola');

-- Enjeksiyon denemesi:
EXECUTE guvenli_giris('admin'--', 'yanlis_parola');
-- Sonuç: 0 satır döner (sorgu başarısız, enjeksiyon etkisiz)

-- Python pycopg2 ile parametre bağlama:
-- cursor.execute(
--     "SELECT * FROM staff WHERE username = %s AND password_hash = crypt(%s, password_hash)",
--     (kullanici_adi, parola)
-- )
```



7.5 Injection Test Karşılaştırma Tablosu

Saldırı Payloadı	Tip	Dinamik SQL Sonucu	Prepared Statement Sonucu
admin'--	Auth Bypass	Giriş BAŞARILI (parola atlanır)	0 satır – Giriş BAŞARISIZ
'; DROP TABLE customer;--	DDL Injection	Tablo SİLİNİR (felakete yol açar)	Hata: sözdizimi hatası yok, tablo güvende
' OR '1'='1	Always-True	TÜM kayıtlar döner	Yalnızca '1'='1' değerli email döner → 0 sonuç
' UNION SELECT username,password_hash...	UNION Extract	Parola hash'leri sızar	UNION ifadesi veri olarak aranır → 0 sonuç
'; UPDATE staff SET password='hacked'--	DML Injection	Parola DEĞİŞTİRİLİR	Sözdizimi hatası değil, güncelleme gerçekleşmez

Ek önlem olarak pg_stat_statements uzantısı ile anormal sorgu desenleri izlenebilir; application_name ile uygulama kaynaklı sorgular tanımlanabilir.

8. AUDIT LOGLARI

8.1 pgaudit Yapılandırması

Tehdit	Varsayılan PostgreSQL loglama yalnızca bağlantı ve hata olaylarını kaydeder. Hangi kullanıcının hangi tabloyu ne zaman sorguladığı bilinmediğinden, yetkisiz erişim olayları tespit edilemez.
Çözüm	pgaudit uzantısı SESSION (oturum) ve OBJECT (nesne) düzeyinde ayrıntılı denetim logu üretir. Bu sayede SQL denetimleri (SQL audit trail) oluşturulur ve uyumluluk gereksinimleri (KVKK, SOX, PCI-DSS) karşılanır.
Uygulama	Bölüm 2.4'teki postgresql.conf değişkenleri uygulandıktan sonra pgaudit.log parametresi konfigüre edilir.

```
-- pgaudit parametrelerini dinamik olarak ayarla
ALTER SYSTEM SET pgaudit.log = 'write, ddl, role, connection';
ALTER SYSTEM SET pgaudit.log_catalog = 'on';
ALTER SYSTEM SET pgaudit.log_parameter = 'on';
ALTER SYSTEM SET pgaudit.log_statement_once = 'off';
ALTER SYSTEM SET pgaudit.role = 'auditor';

-- Değişiklikleri yükle
SELECT pg_reload_conf();

-- Aktif ayarları doğrula
SHOW pgaudit.log;
SHOW pgaudit.log_catalog;

-- Belirli bir kullanıcı için audit etkinleştir
ALTER ROLE editor SET pgaudit.log = 'all';
ALTER ROLE dept_user SET pgaudit.log = 'write';
```

8.2 pgaudit.log Parametreleri

Parametre Değeri	Kapsam	Açıklama
read	DQL	SELECT, COPY FROM ifadeleri
write	DML	INSERT, UPDATE, DELETE, TRUNCATE, COPY TO
function	PL/pgSQL	Fonksiyon çağrıları ve saklı yordam yürütmeleri
role	DCL	GRANT, REVOKE, CREATE/ALTER/DROP ROLE
ddl	DDL	CREATE, ALTER, DROP tablo/index/view işlemleri
misc	Diğer	FETCH, CHECKPOINT, VACUUM gibi çeşitli işlemler
connection	Bağlantı	Oturum açma/kapama olayları
all	Hepsi	Tüm kategoriler (üretim için dikkatli kullanılmalı)

8.3 Örnek Log Çıktısı

```
C:\Users\yildi>docker logs postgres 2>&1 | findstr "AUDIT"
2026-04-21 00:22:05.334 GMT [156] LOG:  AUDIT: SESSION,1,1,ROLE,CREATE ROLE,,,CREATE ROLE db_admin WITH LOGIN PASSWORD <REDACTED>,<not
logged>
2026-04-21 00:22:05.338 GMT [156] LOG:  AUDIT: SESSION,2,1,ROLE,GRANT,SCHEMA,,GRANT ALL ON SCHEMA public TO db_admin,<not logged>
2026-04-21 00:22:05.340 GMT [156] LOG:  AUDIT: SESSION,3,1,ROLE,GRANT,,,GRANT ALL PRIVILEGES ON DATABASE dvdrental TO db_admin,<not log
ged>
2026-04-21 00:22:05.343 GMT [156] LOG:  AUDIT: SESSION,4,1,ROLE,GRANT,TABLE,,GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO db_
admin,<not logged>
2026-04-21 00:22:05.345 GMT [156] LOG:  AUDIT: SESSION,5,1,ROLE,GRANT,SEQUENCE,,GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public
TO db_admin,<not logged>
2026-04-21 00:22:05.348 GMT [156] LOG:  AUDIT: SESSION,6,1,ROLE,GRANT,FUNCTION,,GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public
TO db_admin,<not logged>
2026-04-21 00:22:05.351 GMT [156] LOG:  AUDIT: SESSION,7,1,ROLE,ALTER DEFAULT PRIVILEGES,TABLES,,ALTER DEFAULT PRIVILEGES IN SCHEMA pub
lic GRANT ALL ON TABLES TO db_admin,<not logged>
2026-04-21 00:22:05.356 GMT [156] LOG:  AUDIT: SESSION,8,1,ROLE,CREATE ROLE,,,CREATE ROLE editor WITH LOGIN PASSWORD <REDACTED>,<not lo
gged>
2026-04-21 00:22:05.359 GMT [156] LOG:  AUDIT: SESSION,9,1,ROLE,GRANT,,,GRANT CONNECT ON DATABASE dvdrental TO editor,<not logged>
2026-04-21 00:22:05.361 GMT [156] LOG:  AUDIT: SESSION,10,1,ROLE,GRANT,SCHEMA,,GRANT USAGE ON SCHEMA public TO editor,<not logged>
2026-04-21 00:22:05.364 GMT [156] LOG:  AUDIT: SESSION,11,1,ROLE,GRANT,TABLE,,GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SC
HEMA public TO editor,<not logged>
2026-04-21 00:22:05.367 GMT [156] LOG:  AUDIT: SESSION,12,1,ROLE,GRANT,SEQUENCE,,GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public
TO editor,<not logged>
2026-04-21 00:22:05.370 GMT [156] LOG:  AUDIT: SESSION,13,1,ROLE,REVOKE,SCHEMA,,REVOKE CREATE ON SCHEMA public FROM editor,<not logged>
2026-04-21 00:22:05.376 GMT [156] LOG:  AUDIT: SESSION,14,1,ROLE,CREATE ROLE,,,CREATE ROLE readonly_user WITH LOGIN PASSWORD <REDACTED>
,<not logged>
2026-04-21 00:22:05.379 GMT [156] LOG:  AUDIT: SESSION,15,1,ROLE,GRANT,,,GRANT CONNECT ON DATABASE dvdrental TO readonly_user,<not logg
ed>
2026-04-21 00:22:05.382 GMT [156] LOG:  AUDIT: SESSION,16,1,ROLE,GRANT,SCHEMA,,GRANT USAGE ON SCHEMA public TO readonly_user,<not logge
d>
2026-04-21 00:22:05.384 GMT [156] LOG:  AUDIT: SESSION,17,1,ROLE,GRANT,TABLE,,GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_u
ser,<not logged>
2026-04-21 00:22:05.388 GMT [156] LOG:  AUDIT: SESSION,18,1,ROLE,REVOKE,TABLE,,REVOKE INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA p
ublic FROM readonly_user,<not logged>
2026-04-21 00:22:05.390 GMT [156] LOG:  AUDIT: SESSION,19,1,ROLE,ALTER DEFAULT PRIVILEGES,TABLES,,ALTER DEFAULT PRIVILEGES IN SCHEMA pu
blic GRANT SELECT ON TABLES TO readonly_user,<not logged>
2026-04-21 00:22:05.393 GMT [156] LOG:  AUDIT: SESSION,20,1,ROLE,CREATE ROLE,,,CREATE ROLE dept_user WITH NOLOGIN NOSUPERUSER,<not logg
ed>
2026-04-21 00:22:05.395 GMT [156] LOG:  AUDIT: SESSION,21,1,ROLE,GRANT,,,GRANT CONNECT ON DATABASE dvdrental TO dept_user,<not logged>
```

9. BULGULAR VE SONUÇLAR

9.1 Uygulanan Güvenlik Katmanlarının Özeti

Bu proje kapsamında PostgreSQL dvdrental veritabanı üzerinde birbirini tamamlayan 7 güvenlik katmanı başarıyla tasarlanmış ve uygulanmıştır:

#	Güvenlik Katmanı	Uygulanan Araç/Yöntem	Koruma Sağladığı Tehdit
1	Kimlik Doğrulama	SCRAM-SHA-256, pg_hba.conf	Brute-force, plain-text parola
2	Erişim Denetimi	RBAC (CREATE ROLE, GRANT/REVOKE)	Yetkisiz veri erişimi, ayrıcalık yükseltme
3	Ağ Güvenliği	IP kısıtlama, SSL/TLS 1.3	Ağ saldırıları, MITM, yetkisiz bağlantı
4	Veri Şifreleme	pgcrypto, AES-256, Blowfish bcrypt	Veri sızıntısı, yedek çalınması
5	Satır Düzeyinde Güvenlik	RLS politikaları	Yatay ayrıcalık yükseltme
6	SQL Enjeksiyon Önleme	Prepared Statements, parametrelili sorgu	Veritabanı manipülasyonu, veri sızıntısı
7	Denetim Loglama	pgaudit, CSV log, log analizi	İz sürememe, uyumsuzluk, iç tehdit

9.2 Güvenlik Öncesi / Sonrası Karşılaştırma

Güvenlik Boyutu	Güvenlik Öncesi	Güvenlik Sonrası
Kullanıcı yönetimi	Tek postgres süper kullanıcı	4 farklı rol, granüler izinler
Parola saklama	MD5 (kırılabılır)	SCRAM-SHA-256 (güvenli)
Hassas veri	Düz metin TEXT sütunları	AES-256 şifreli BYTEA
Veri izolasyonu	Tüm tablolar herkese açık	RLS ile kullanıcı bazlı filtreleme
SQL enjeksiyon	Dinamik sorgu – savunmasız	Prepared Statement – güvenli
Olay kaydı	Yalnızca bağlantı hataları	pgaudit ile tüm SQL işlemleri
Uyumluluk (KVKK/GDPR)	Risk altında	Veri şifreleme + log = uyumlu
Yedek güvenliği	Düz metin veri içerir	Şifreli veri – anlamsız
SSL/TLS	Zorunlu değil	TLS 1.3 zorunlu, 1.0/1.1 devre dışı

9.3 Genel Değerlendirme

Bu proje, PostgreSQL'in yerleşik güvenlik mekanizmalarının doğru yapılandırıldığında endüstriyel düzeyde bir koruma sağladığını ortaya koymaktadır. Uygulanan katmanlı güvenlik (defense-in-depth) yaklaşımı; tek bir güvenlik açığının sistemi tamamen tehlikeye atmasının önüne geçmektedir. Özellikle:

- RLS politikaları, veri izolasyonunu uygulama katmanına bırakmak yerine veritabanı motorunun kendisinde zorunlu kılmakta; bu sayede uygulama kodundaki hatalar bile veri sızıntısına yol açamamaktadır.
- pgcrypto ile sütun düzeyinde şifreleme, veritabanı yedeğinin çalınması senaryosunda kritik bir güvence katmanı oluşturmaktadır.
- pgaudit denetim logları, KVKK Madde 12 kapsamındaki teknik tedbirler ile ISO 27001 erişim kontrolü gereksinimlerini karşılamaktadır.
- Prepared Statement kullanımı, OWASP Top 10'un 1 numaralı tehdit kategorisi olan SQL Injection'ı tamamen engeller.

İleride yapılabilecek iyileştirmeler arasında; HashiCorp Vault ile merkezi anahtar yönetimi, pg_partman ile log arşivleme otomasyonu ve Patroni ile yüksek erişilebilirlik kümesi kurulumu sayılabilir.